RISC-V: Processor Design: Datapath

Computer Architecture

Dr. Mahmoudi

April 15, 2025

Outline

- Introduction
- Logic Design Conventions
- Datapath elements
- Building a datapath
- The Control

Recall: Performance

- The performance of a computer is determined by three key factors: instruction count, clock cycle time, and clock cycles per instruction (CPI).
- The compiler and the instruction set architecture determine the instruction count required for a given program.
- The implementation of the processor determines both the clock-cycle time and the number of clock cycles per instruction.

Recall: Levels of Representation/Interpretation

High Level Language Program (e.g., C)

```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

Compiler

Assembly Language Program (e.g., RISC-V)

```
lw    x3, 0(x10)
lw    x4, 4(x10)
sw    x4, 0(x10)
sw    x3, 4(x10)
```

Assembler

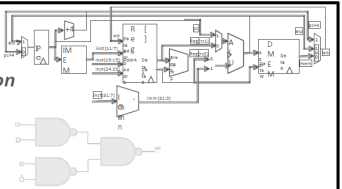
Machine Language Program (RISC-V)

```
1000 1101 1110 0010 0000 0000 0000 0000
1000 1110 0001 0000 0000 0000 0000 0100
1010 1110 0001 0010 0000 0000 0000 0000
1010 1101 1110 0010 0000 0000 0000 0100
```

Hardware Architecture Description (e.g., block diagrams)

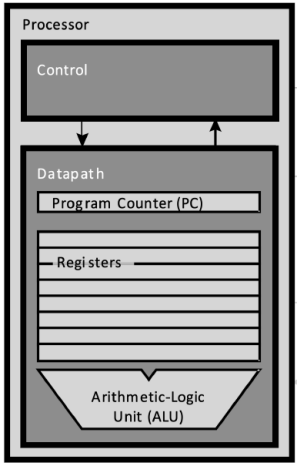
Architecture Implementation

Logic Circuit Description (Circuit Schematic Diagrams)



Recall: Single-Core Processor

- **Datapath (“the brawn”)**: portion of the processor that contains hardware necessary to perform operations required by the processor.
- **Control (“the brain”)**: portion of the processor that tells the datapath what needs to be done.



Logic Design Conventions: combinational elements

- The datapath elements in the RISC-V implementation consist of two different types of logic elements:
- Elements that operate on data values and elements that contain state.
  - The elements that operate on data values are all **combinational**, which means that their outputs depend only on the current inputs.
  - Given the same input, a combinational element always produces the same output because it has no internal storage.
  - Examples: AND gate or an ALU.

Logic Design Conventions: state/sequential elements

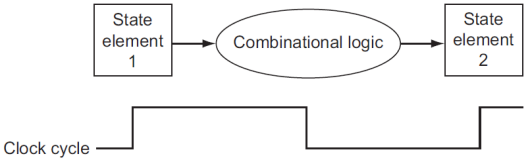
- An element contains state if it has some internal storage.
- We call these elements **state elements** because we could restart the computer accurately by loading the state elements with the values they contained before.
- Examples: instruction and data memories, registers.
- A state element has at least two inputs and one output. The required inputs are the data value to be written into the element and the clock, which determines when the data value is written.
- The output provides the value that was written in an earlier clock cycle.
- Also called sequential, because their outputs depend on both their inputs and the contents of the internal state.

Logic Design Conventions: Clocking Methodology

- A clocking methodology defines when signals can be read and when they can be written.
- **Important**, because if a signal is written at the same time that it is read, the value of the read could correspond to the old value, the newly written value, or even some mix of the two!
- **Clocking methodology**: The approach used to determine when data are valid and stable relative to the clock.

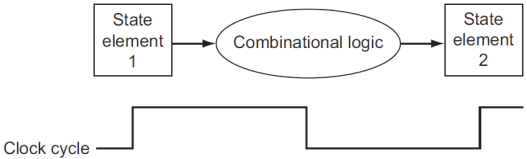
Logic Design Conventions: Clocking Methodology

- An edge-triggered clocking methodology means that any values stored in a sequential logic element are updated only on a clock edge.
- It is a quick transition from low to high or vice versa.



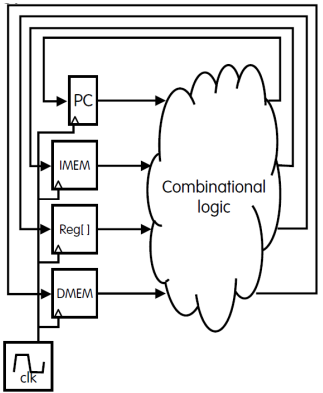
Logic Design Conventions: Clocking Methodology

- Only state elements can store a data value, thus any collection of combinational logic must have its inputs come from a set of state elements and its outputs written into a set of state elements.
- The inputs are values that were written in a previous clock cycle, while the outputs are values that can be used in a following clock cycle.



One-Instruction-Per-Cycle RISC V Machine

- The CPU is composed of two types of subcircuits: **combinational logic blocks** and **state elements**.
- On every tick of the clock, the computer executes one instruction:
  - Current outputs of the state elements drive the inputs to combinational logic.
  - whose outputs settle at the inputs to the state elements before the next rising clock edge.
- At the rising clock edge:
  - All the state elements are updated with the combinational logic outputs.
  - and execution moves to the next clock cycle.



ensia

Datapath elements

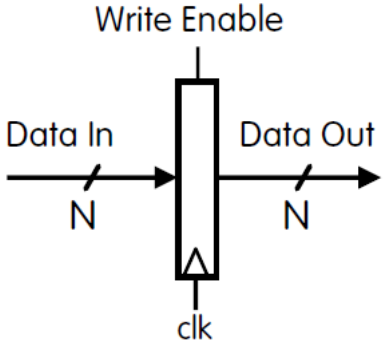
Datapath elements

- **Datapath element:** A unit used to operate on or hold data within a processor.
- In the RISC-V implementation, the datapath elements include:
  1. Instruction memory.
  2. Data memory.
  3. Register file.
  4. ALU.
  5. Adders.
  6. PC.

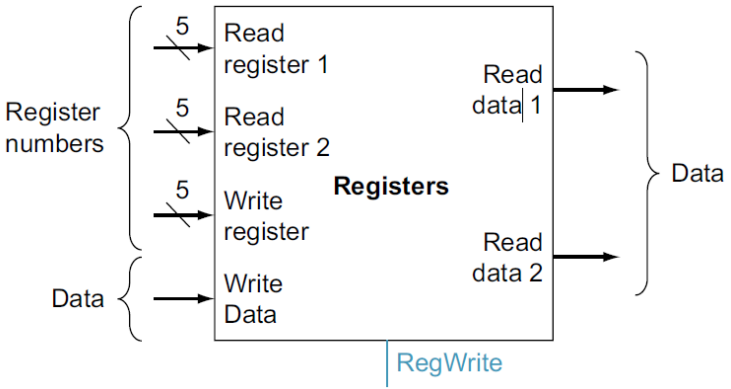
Building a Datapath: State Elements

program counter (PC), 32-bit register that holds the address of the current instruction.

- **Input:**
  - N-bit data input bus,
  - Write Enable “Control” bit (1: asserted/high, 0: deasserted/0)
- **Output:**
  - N-bit data output bus.
- **Behavior:**
  - If Write Enable is 1 on the rising clock edge, set Data Out=Data In.
  - At all other times, Data Out will not change; it will output its current value.

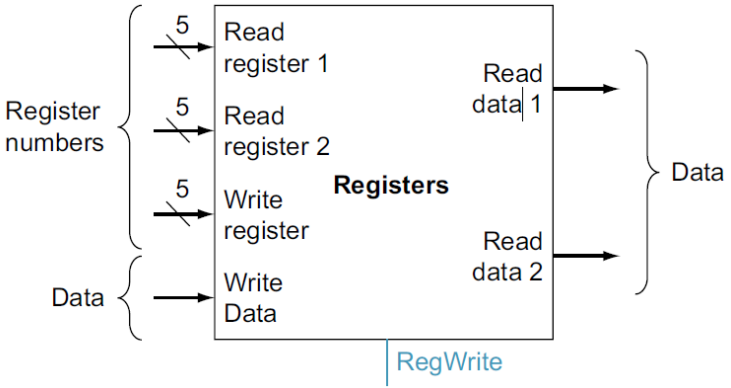


Building a Datapath: State Elements



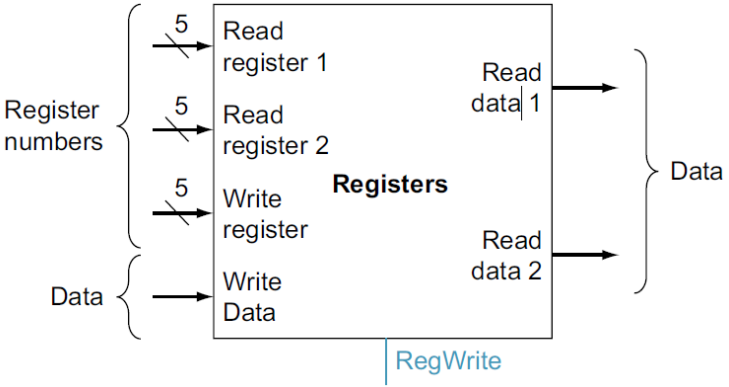
- The processor's 32 general-purpose registers are stored in a structure called a **register file**.
- A **register file** is a collection of registers in which any register can be read or written by specifying the number of the register in the file.

Building a Datapath: State Elements



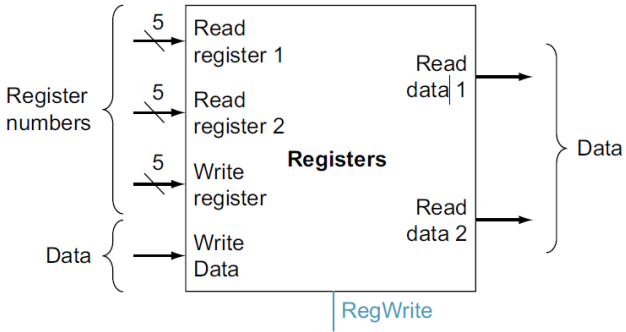
- For each data word to be read from the registers, we need an input to the register file that specifies the register number to be read and an output from the register file that will carry the value that has been read from the registers.

Building a Datapath: State Elements



- To write a data word, we will need two inputs: one to specify the register number to be written and one to supply the data to be written into the register.

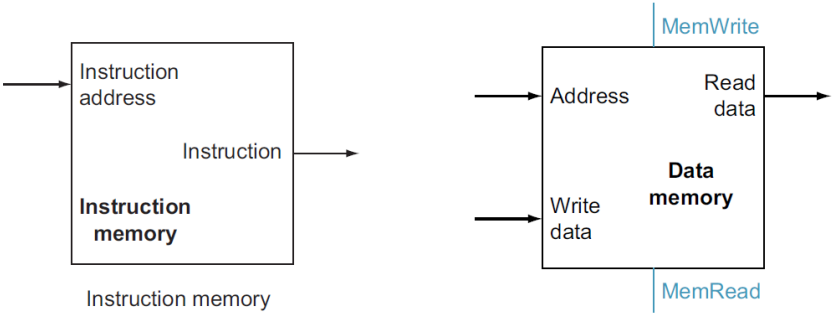
Building a Datapath: State Elements



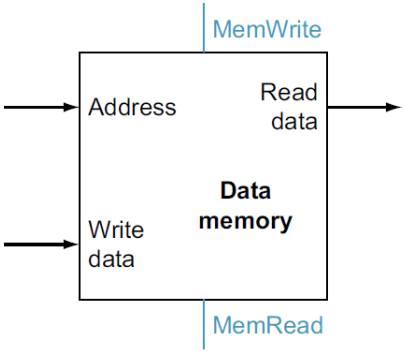
- The register file always outputs the contents of whatever register numbers are on the Read register inputs.
- Writes, however, are controlled by the write control signal, which must be asserted for a write to occur at the clock edge.

Building a Datapath: State Elements

- In our processor, we'll use two "separate" memories:
  1. **IMEM**: A read-only memory for fetching instructions.
  2. **DMEM**: A memory for loading (read) and storing (write) data words.



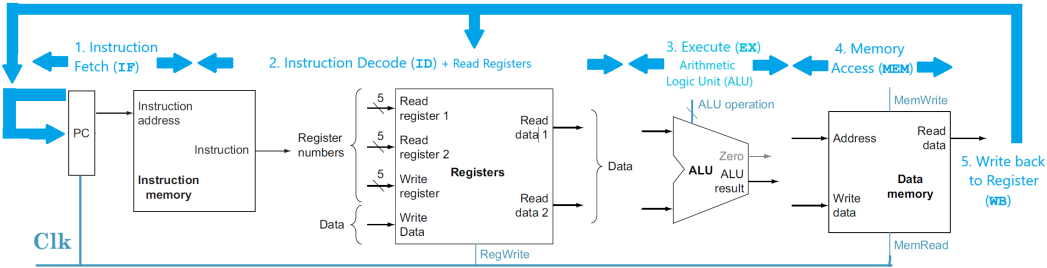
Building a Datapath: State Elements



Data memory unit

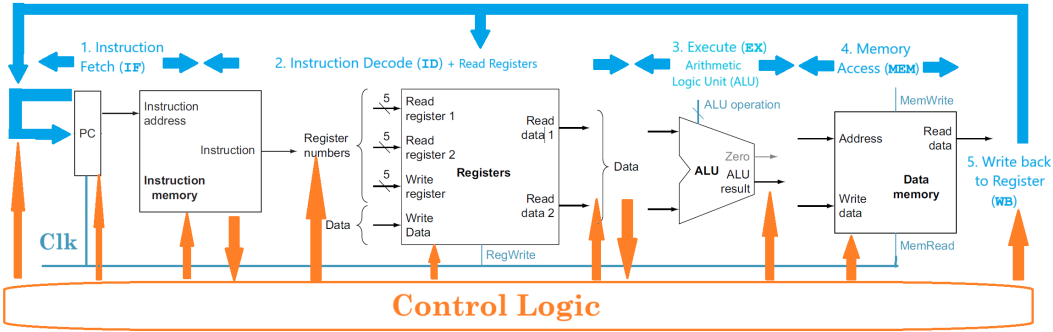
- Memory words are accessed as follows:
  - **Read**: Set MemRead=1.
    Address selects word to put on Read data bus.
  - **Write**: Set MemWrite=1.
    Address selects word to be written with Write data bus.
- Like Register File, clock input is only a factor on write (write occurs on rising clock edge).

Five Basic Stages (Phases) of Instruction Execution



Five Basic Stages (Phases) of Instruction Execution

- Not All Instructions Need All five Stages!
- The control logic selects “needed” datapath lines based on the instruction.



ensia

Building a datapath

A Basic RISC-V Implementation

We will be examining an implementation that includes a subset of the core RISC-V instruction set:

- The memory-reference instructions load word (lw) and store word (sw).
- The arithmetic-logical instructions add, sub.
- The conditional branch instruction branch if equal (beq).

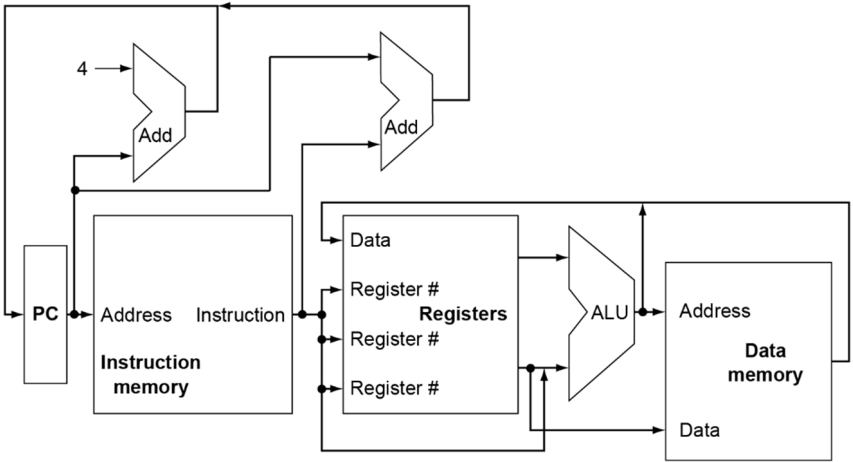
An Overview of the Implementation

For every instruction, the first two steps are identical:

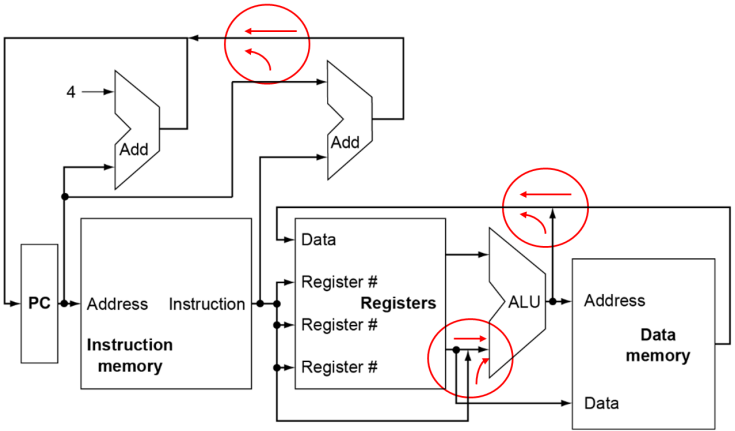
1. Send the **program counter (PC)** to the memory that contains the code and fetch the instruction from that memory.
2. Read one or two registers, using fields of the instruction to select the registers to read. For the lw instruction, we need to read only one register, but most other instructions require reading two registers.

After these two steps, the actions required to complete the instruction depend on the instruction class.

An Overview of the Implementation

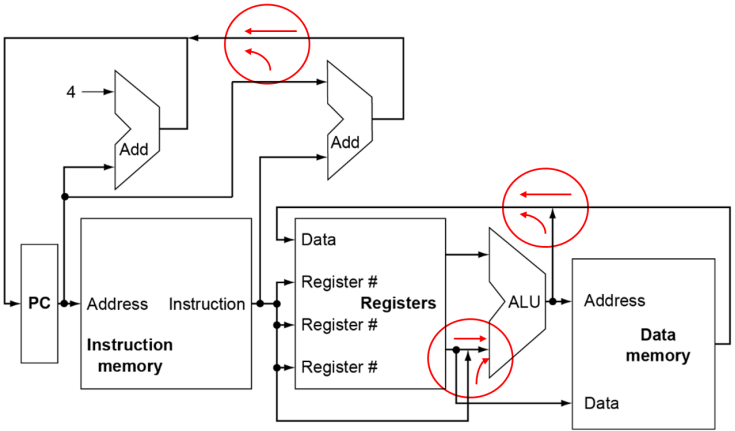


An Overview of the Implementation



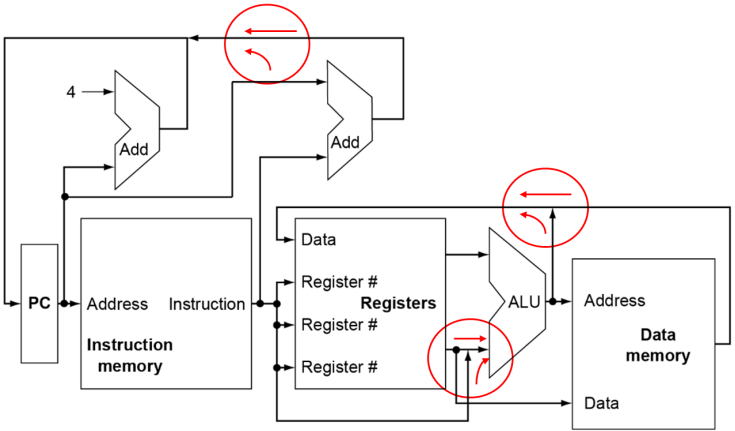
- Can’t just join wires together
- Use multiplexers

An Overview of the Implementation



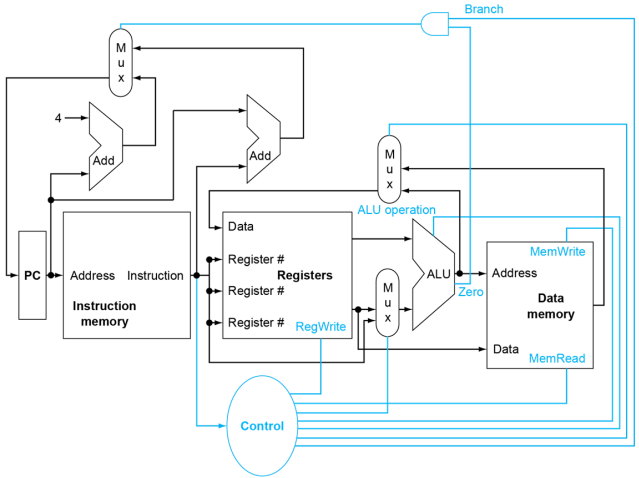
- Several of the units must be controlled depending on the type of instruction.
- **For example**, the data memory must read on a load and write on a store.

An Overview of the Implementation



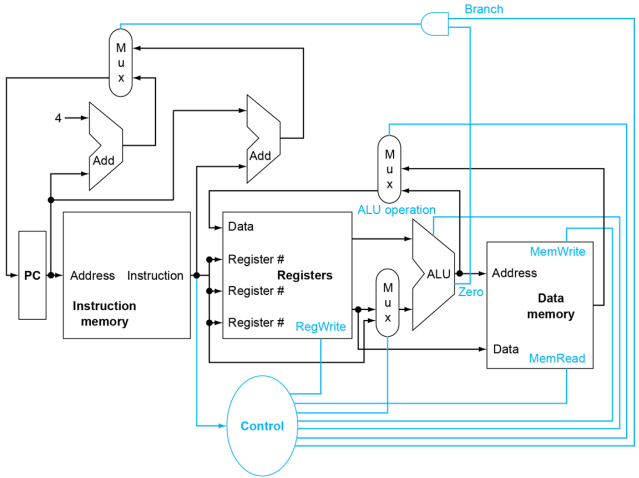
- The register file must be written only on a load or an arithmetic-logical instruction.
- And, of course, the ALU must perform one of several operations.
- **We need some Control!!**

Control



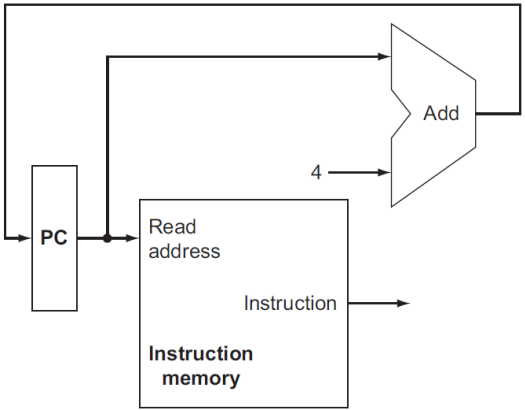
- A **control unit**, which has the instruction as an input, is used to determine how to set the control lines for the functional units and two of the multiplexors.

Control



- The top multiplexor, which determines whether **PC + 4** or the **branch destination address** is written into the **PC**
- It is set based on the **Zero** output of the ALU, which is used to perform the comparison of a **beq** instruction.

Building a Datapath

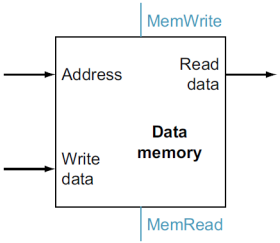


- To execute any instruction, we must start by fetching the instruction from memory.
- To prepare for executing the next instruction, we must also increment the program counter so that it points at the next instruction, 4 bytes later.

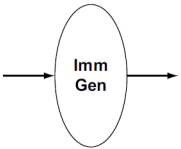
Building a Datapath (Load and Store)

- These instructions compute a memory address by adding the base register to the 12-bit signed offset field contained in the instruction.
- If the instruction is a store, the value to be stored must also be read from the register file where it resides.
- If the instruction is a load, the value read from memory must be written into the register file in the specified register.
- Thus, we will need both the register file and the ALU.

Building a Datapath (Load and Store)



a. Data memory unit



b. Immediate generation unit

- Furthermore, we will need a unit to sign-extend the 12-bit offset field in the instruction to a 32-bit signed value.
- A data memory unit to read from or write to.
- The data memory must be written on store instructions; hence, data memory has read and write control signals, an address input, and an input for the data to be written into memory.

Building a Datapath (Load)

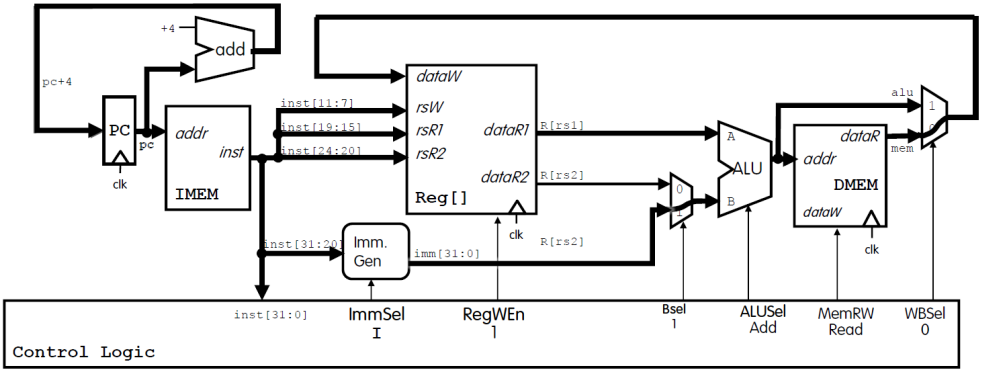
Increment PC to next instruction.

Immediate Generation Block builds a 32-bit immediate **`imm`**.

ALU computes address **`alu = R[rs1] + imm`**.

Read memory at address **`alu`**.

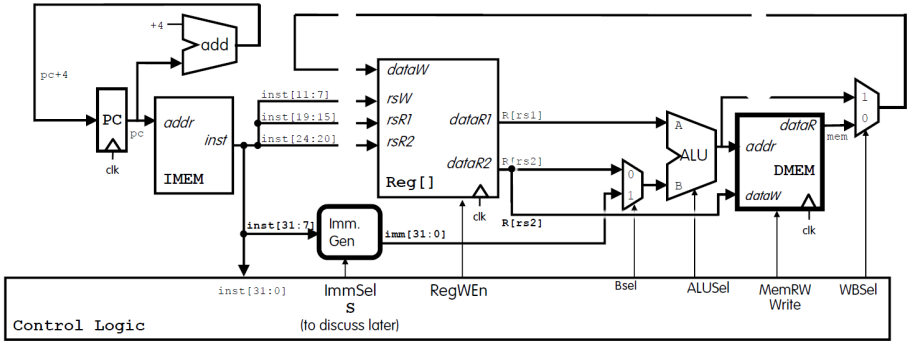
Write loaded memory value **`mem`** to register.



Building a Datapath (Store)

Control ImmSel selects how to generate immediate type: **`I`**, **`S`**.

Control MemRW=Write saves **`rs2`** to memory on the next rising clock edge.



Building a Datapath (beq)

- The beq instruction has three operands, two registers that are compared for equality, and a 12-bit offset used to compute the branch target address relative to the branch instruction address.
- **Branch target:** *address The address specified in a branch, which becomes the new program counter (PC) if the branch is **taken**. In the RISC-V architecture, the branch target is given by the sum of the offset field of the instruction and the address of the branch*.

Building a Datapath (beq)

- There are two details in the definition of branch instructions to which we must pay attention:
  1. The instruction set architecture specifies that the base for the branch address calculation is the address of the branch instruction.
  2. The architecture also states that the offset field is shifted left 1 bit so that it is a half word offset; this shift increases the effective range of the offset field by a factor of 2.
- To deal with the latter complication, we will need to shift the offset field by 1.

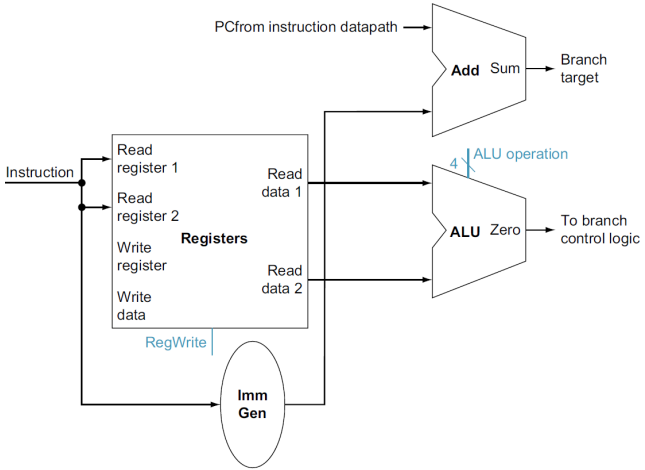
Building a Datapath (beq)

- As well as computing the branch target address, we must also determine whether the next instruction is the instruction that follows sequentially or the instruction at the branch target address.
- When the condition is true (i.e., two operands are equal), the branch target address becomes the new PC, and we say that the **branch is taken**.
- **Branch taken:** *A branch where the branch condition is satisfied and the program counter (PC) becomes the branch target. All unconditional branches are taken branches*.

Building a Datapath (beq)

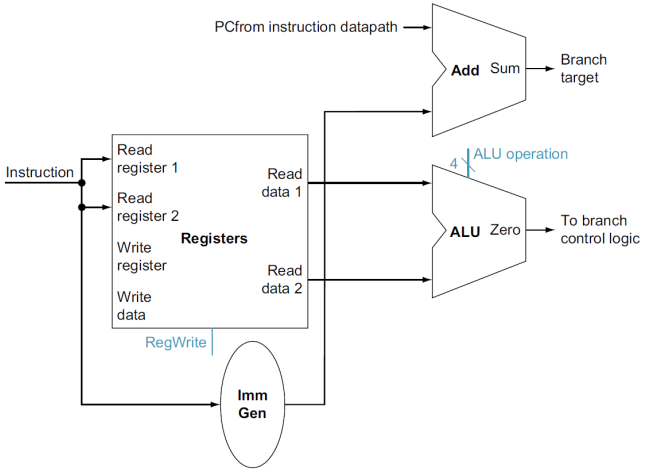
- If the operand is not zero, the incremented PC should replace the current PC (just as for any other normal instruction); in this case, we say that the **branch is not taken**.
- **branch not taken or (untaken branch)** *A branch where the branch condition is false and the program counter (PC) becomes the address of the instruction that sequentially follows the branch*.

Building a Datapath (beq)



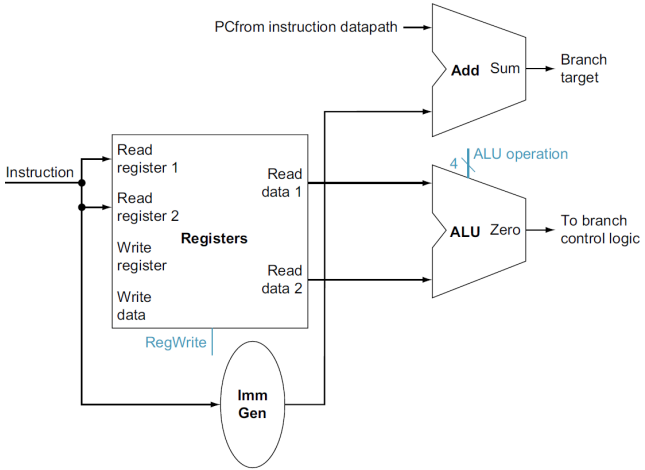
- To compute the branch target address, the branch datapath includes an immediate generation unit and an adder.
- To perform the compare, we need to use the register file to supply two register operands.

Building a Datapath (beq)



- In addition, the equality comparison can be done using the ALU.
- Since that ALU provides an output signal that indicates whether the result was 0, we can send both register operands to the ALU with the control set to subtract two values.

Building a Datapath (beq)



- If the Zero signal out of the ALU unit is asserted, we know that the register values are equal.
- The branch instruction operates by adding the PC with the 12 bits of the instruction shifted left by 1 bit.
- Simply concatenating 0 to the branch offset accomplishes this shift.

Building a Datapath (Branch)

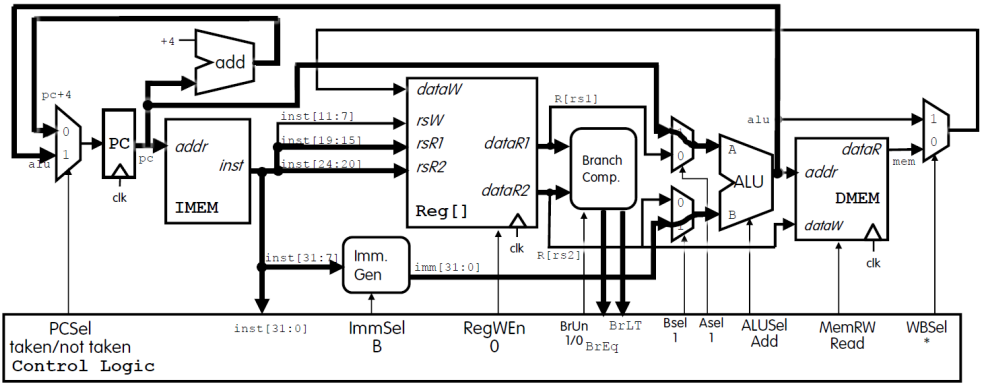
If PCSel=taken, update PC to ALU output. Else, update to next instruction `PC + 4`.

Build `imm` from B-type instruction.

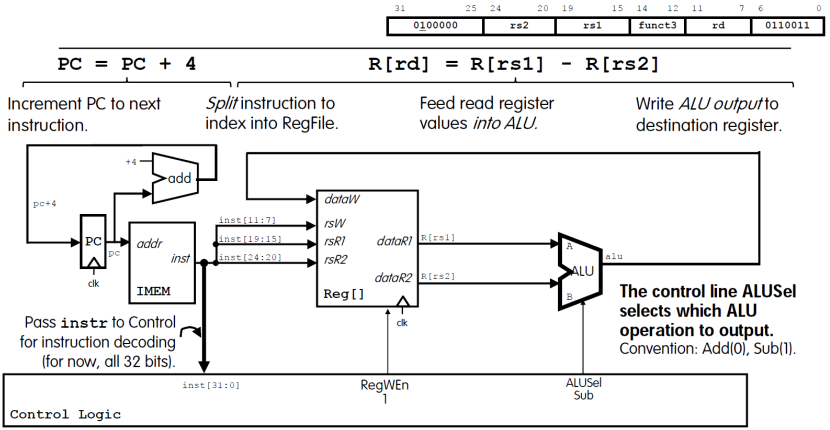
- Compute branch; feed to Control.
- Compute `PC + imm`.

Don't write to memory.

Don't write to registers.



Building a Datapath (add/sub)



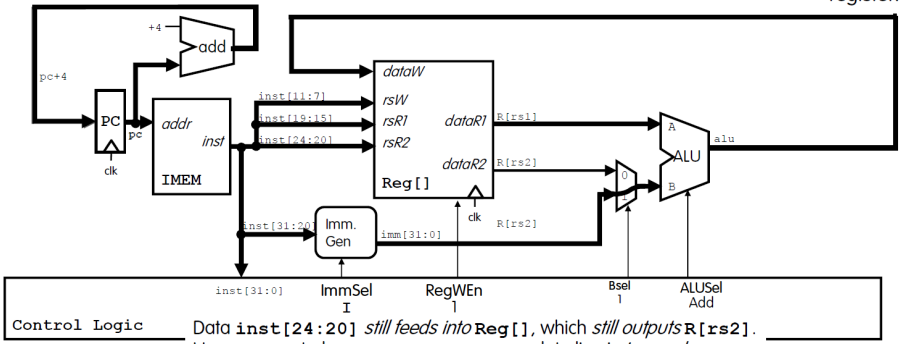
Building a Datapath (addi)

Increment PC to next instruction.

Immediate Generation Block builds a 32-bit immediate **`imm`** from instruction bits.

Control line Bsel=1 selects the generated immediate **`imm`** for ALU input B.

Write ALU output to destination register.



Data **`inst[24:20]`** *still feeds into* **`Reg[]`**, which *still outputs* **`R[rs2]`**. However, control **`Bsel=1`** means **`R[rs2]`** data line is *ignored*.

Building a Datapath (jal)

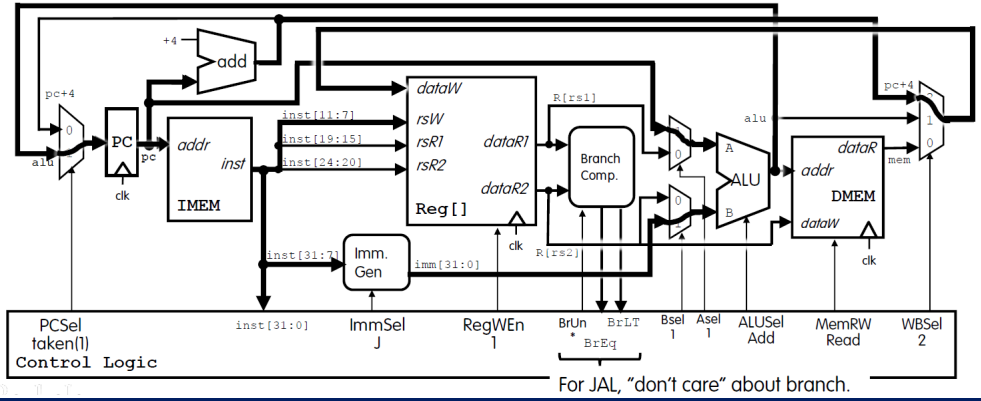
- Feed PC into blocks.
- Write ALU output to PC.

Generate byte offset `imm` for 20-bit PC-relative jump.

Compute `PC + imm`.

Write `PC + 4` to destination register.

Don't write to memory.



Building a Datapath (jalr)

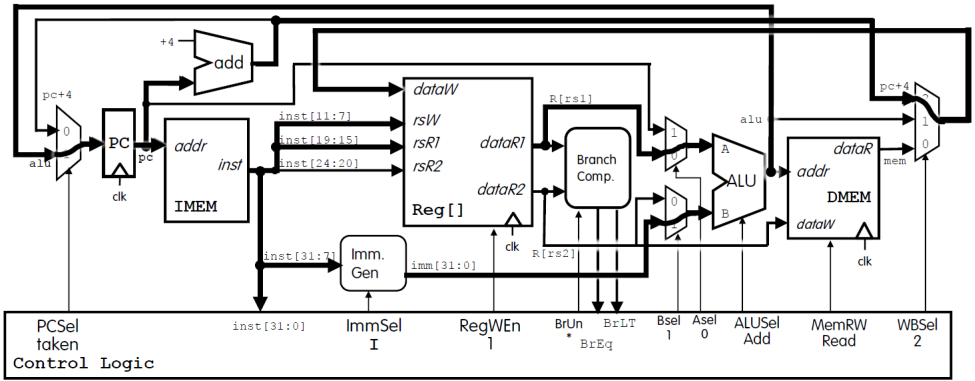
- Feed PC into blocks.
- Write ALU output to PC.

Generate 12-bit `imm` (`I`-Format).

Compute `rs1 + imm`.

Write `PC + 4` to destination register.

Don't write to memory.



Building a Datapath (lui)

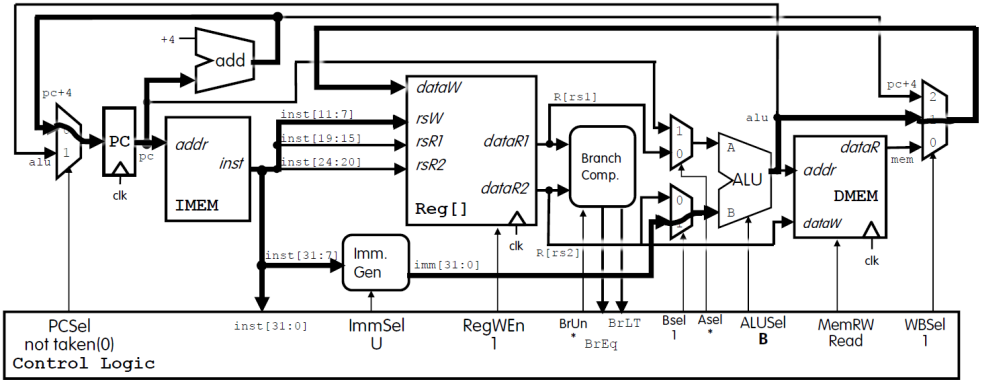
Increment PC to next instruction.

Generate `imm` with upper 20 bits. (U-format)

Grab only `imm`! (ALUSel=B)

Write result to destination register.

Don't write to memory.



Building a Datapath (auipc)

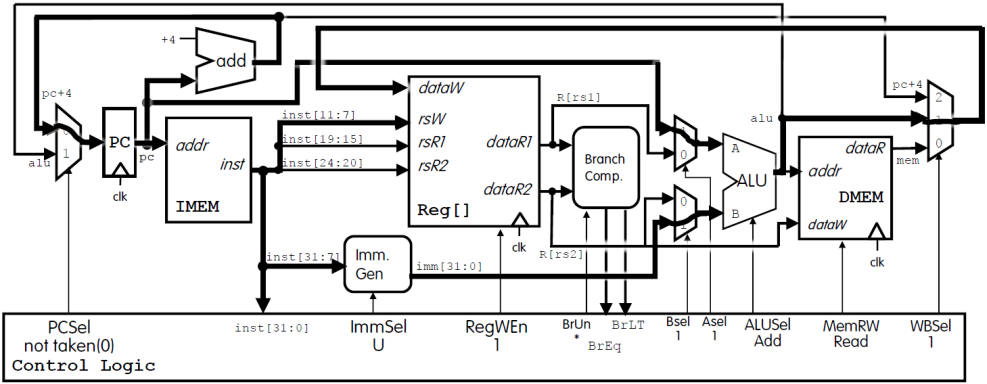
Increment PC to next instruction.

Generate `imm` with upper 20 bits. (U-format)

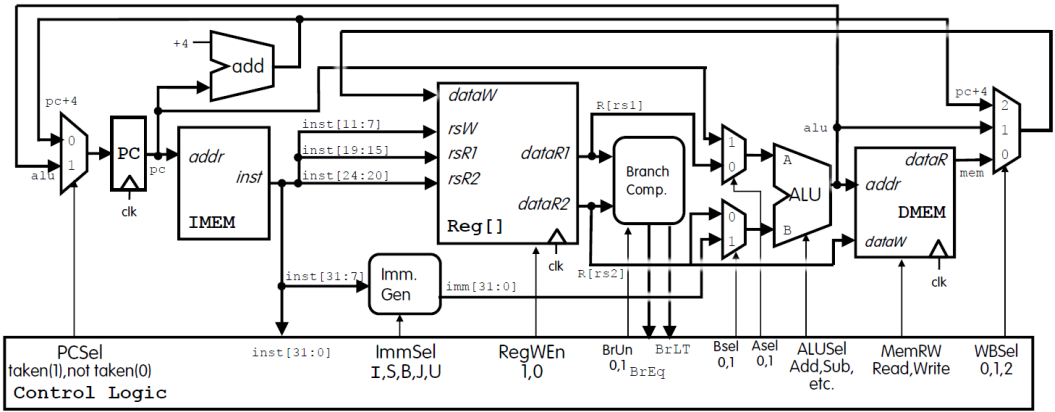
Add `PC + imm`!

Write result to destination register.

Don't write to memory.



Complete RV31I Datapath



The Control: A Simple Implementation Scheme

- The control unit must be able to take inputs and generate a write signal for each state element, the selector control for each multiplexor, and the ALU control.
- This simple implementation covers load word (lw), store word (sw), branch if equal (beq), and the arithmeticlogical instructions add, sub, and, and or.

The ALU Control

- For load and store instructions, we use the ALU to compute the memory address by addition.
- For the R-type instructions (AND, OR, add, or subtract), depending on the value of the 7-bit funct7 field (bits 31:25) and 3-bit funct3 field (bits 14:12) in the instruction.
- For the conditional branch if equal instruction, the ALU subtracts two operands and tests to see if the result is 0.

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract

The ALU Control

We can generate the 4-bit ALU control input using a small control unit that has as inputs the funct7 and funct3 fields of the instruction and a 2-bit control field, which we call ALUOp.

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
lw	00	load word	XXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

The ALU Control

ALUOp indicates whether the operation to be performed should be add (00) for loads and stores, subtract and test if zero (01) for beq, or be determined by the operation encoded in the funct7 and funct3 fields (10).

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
lw	00	load word	XXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

The ALU Control

The output of the ALU control unit is a 4-bit signal that directly controls the ALU by generating one of the 4-bit combinations.

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
lw	00	load word	XXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

The ALU Control

Only a small number of the possible funct field values are of interest and funct fields are used only when the ALUOp bits equal 10

ALUOp		Funct7 field							Funct3 field			
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	Operation
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

The ALU Control

Some don't-care entries have been added. For example, the ALUOp does not use the encoding 11, so the truth table can contain entries 1X and X1, rather than 10 and 01.

ALUOp		Funct7 field							Funct3 field			
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	Operation
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

The ALU Control

While we show all 10 bits of funct fields, note that the only bits with different values for the four R-format instructions are bits 30, 14, 13, and 12. Thus, we only need these four funct field bits as input for ALU control instead of all 10.

ALUOp		Funct7 field							Funct3 field			
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	Operation
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

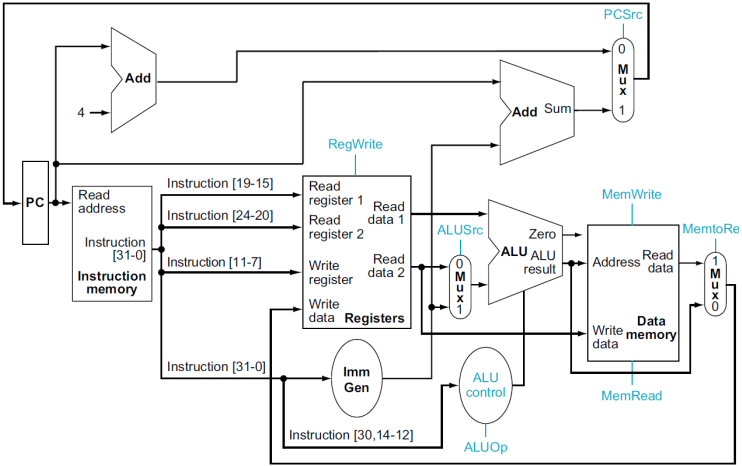
The four instruction classes

The opcode field is always in bits 6:0, funct3 14:12, funct7 31:25, rs1 19:15, rs2 24:20, rd 11:7.

Name (Bit position)	Fields					
	31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

Designing the Main Control Unit

- The ALU control block depends on the funct3 field and part of the funct7 field.
- The PC does not require a write control, since it is written once at the end of every clock cycle;



Designing the Main Control Unit

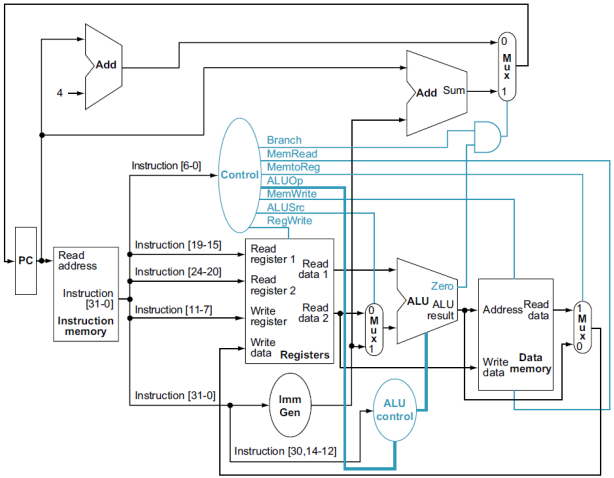
Signal name	Effect when deasserted	Effect when asserted
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, 12 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of PC + 4.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

Designing the Main Control Unit

- The control unit can set all but one of the control signals based solely on the opcode and funct fields of the instruction.
- The PCSrc control line is the exception.
- That control line should be asserted if the instruction is a branch if equal (a decision that the control unit can make) and the Zero output of the ALU, which is used for the equality test, is asserted.
- To generate the PCSrc signal, we will need to AND together a signal from the control unit, which we call Branch, with the Zero signal out of the ALU.

Designing the Main Control Unit

- Remember that the state elements all have the clock as an implicit input and that the clock is used in controlling writes.
- The input to the control unit is the 7-bit opcode field.
- The output: six 1-bit signals (ALUSrc, MemtoReg, RegWrite, MemRead, MemWrite, and Branch) and one 2-bit signal (ALUOp).

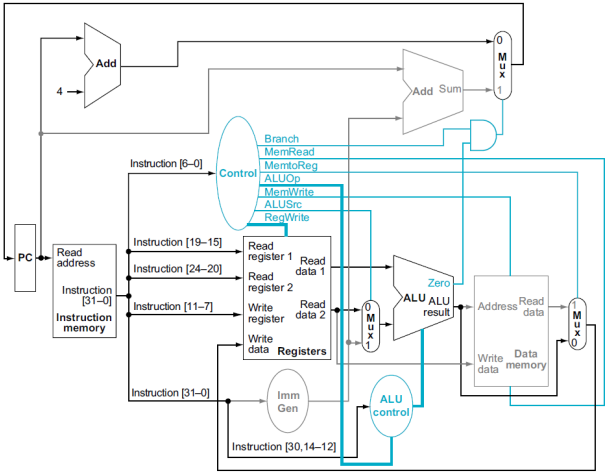


Designing the Main Control Unit

Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

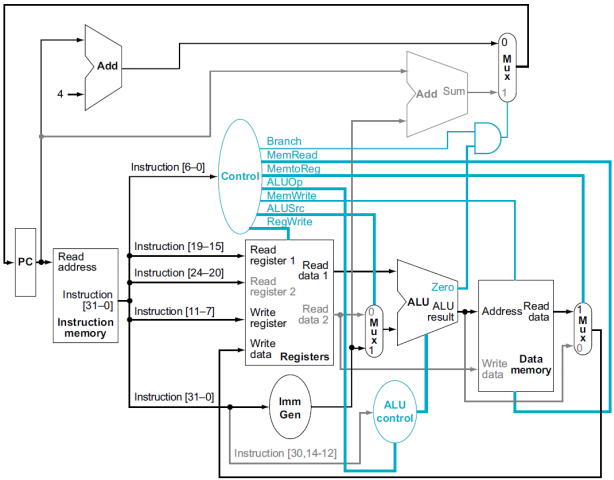
Designing the Main Control Unit: R-type

1. The instruction is fetched, and the PC is incremented.
2. Read rs1 and rs2 from the register file; also, the main control unit computes the setting of the control lines during this step.
3. The ALU operates on the data read from the register file.
4. The result is written into the destination register (rd) in the register file.



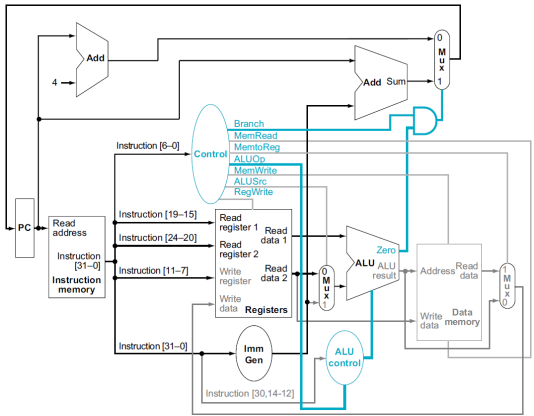
Designing the Main Control Unit: Load

1. Instruction fetched, and the PC incremented.
2. A register (rs1) value is read.
3. The ALU sums of the value in rs1 and the sign-extended 12 bits (offset).
4. The sum from the ALU is used as the address for the data memory.
5. The data from the memory unit is written into the register file (rd).



Designing the Main Control Unit: beq

1. Instruction fetched, and the PC incremented.
2. Two registers are read from the register file.
3. The ALU subtracts one data value from the other data value. The value of PC is added to the sign-extended, 12-bit offset left shifted by one;
4. The Zero status of the ALU is used to decide which adder result to store in the PC.



Finalizing Control

- The outputs are the control lines, and the inputs are the opcode bits.

Input or output	Signal name	R-format	lw	sw	beq
Inputs	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Outputs	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

Why a Single-Cycle Implementation is not Used Today

- Notice that the clock cycle must have the same length for every instruction in this single-cycle design.
- The longest possible path in the processor determines the clock cycle (load instruction).
- Although the CPI is 1, the overall performance of a single-cycle implementation is likely to be poor since the clock cycle is too long.
- Historically, early computers with very simple instruction sets did use this implementation technique.
- Useless to try implementation techniques that reduce the delay of the common case but do not improve the worst-case cycle time. (**making the common case fast**).

ensia

Thank you!